



HSMU-SpGEMM: Achieving High Shared Memory Utilization for Parallel Sparse General Matrix-Matrix Multiplication on Modern GPUs

¹Min Wu, ¹Huizhang Luo*, ¹Fenfang Li, ²Yiran Zhang, ¹Zhuo Tang, ¹Kenli Li, ²Jeff Zhang, ¹Chubo Liu

¹Hunan University, Changsha, China

²Arizona State University, Tempe, United States

minwuq@hnu.edu.cn, luohuizhang@hnu.edu.cn, fenfangli@hnu.edu.cn, yzha1257@asu.edu,

ztang@hnu.edu.cn, lkl@hnu.edu.cn, jeffzhang@asu.edu, liuchubo@hnu.edu.cn

Abstract—Sparse general matrix-matrix multiplication (SpGEMM) is a core primitive for numerous scientific applications. Traditional hash-based approaches fail to strike a balance between reducing hash collisions and efficiently utilizing fast shared memory, which significantly undermines the performance of executing SpGEMM on GPUs. To address this issue, this paper introduces a novel accumulator design that achieves high shared memory utilization on modern GPUs. For the proposed high shared memory utilization algorithm, i.e., HSMU-SpGEMM¹, we further optimize different symbolic stages. Our evaluations with four state-of-the-art hash-based SpGEMM libraries (*Nsparse*, *spECK*, *OpSparse*, and NVIDIA's *cuSPARSE*) on three NVIDIA GPUs (Ampere, Ada Lovelace, Turing) demonstrate significant performance benefits from HSMU-SpGEMM.

Index Terms—SpGEMM, Sparse matrix, GPU, High shared memory utilization, Novel accumulator

I. INTRODUCTION

Sparse general matrix-matrix multiplication (SpGEMM) refers to multiplying two sparse matrices A and B to produce a sparse output matrix C . This computational kernel is a core primitive in various numerical applications such as scientific simulations and graph processing, including breadth-first search [1], Markov clustering [2], [3], algebraic multigrid solvers [4], [5] and triangular counting [6], [7], etc.

Gustavson's algorithm [8] has been a popular solution for implementing fast and parallel SpGEMM on GPU architectures. In general, the implementation of Gustavson's algorithm for SpGEMM consists of *symbolic* stage and *numeric* stage [9]–[13]. In the symbolic stage, the primary task is to determine the number of non-zeros (NNZ) in matrix C , which entails pre-allocation of memory for non-zero (NZ) elements before the numeric stage. The numeric stage performs the multiplication and accumulation of intermediate products, computed on the memory allocated during the symbolic stage. The numeric stage is the most time-consuming part of the SpGEMM and it relies on efficient accumulator design [10].

Accumulator design for SpGEMM has been proposed in prior works [10], [12]–[21]. Among them, hash-based accumulators emerge as a promising option due to their low memory requirements and have been adopted in well-known SpGEMM GPU libraries including *Nsparse* [10], *spECK* [12], *OpSparse* [13], and *cuSPARSE* [19], [20]. In these hash-based accumulator libraries, intermediate products are accumulated by mapping them to the hash table using a hash function. At runtime, hash tables are preferably placed in fast but precious GPU's shared memory, as storing them in the larger global memory would result in significant performance degradation. For this reason, the size of the hash table inherently trade-offs between hash collision rate and GPU shared memory utilization² for hash-based accumulators — larger hash tables can reduce hash collisions and improve the efficiency of hash-based accumulators. However, as we show in this paper, naively increasing hash tables may result in lower GPU shared memory utilization.

Drawbacks of state-of-the-art SpGEMM libraries *Nsparse* only sets the length of its hash table to the maximum NNZ of its allocated rows [10] according to each kernel function. This method could theoretically achieve a higher shared memory utilization on GPUs, but also incurs a high hash collision rate that significantly degrades the overall performance. *spECK*, instead, allocate $1.5\times$ of the required memory space for the hash table to reduce the hash collision rate and improve the performance [12]. However, only up to 66% of the final hash table is occupied, resulting in wasted memory resources. *OpSparse* concludes that for best performance the length of the hash table should be set to $2\times$ of the maximum NNZ of its allocated rows [13], which caps the maximum shared memory utilization to be 50%.

In this work, we seek to achieve to best possible SpGEMM performance without wasting GPUs' shared memory. In particular, we designed and implemented a high shared memory utilization algorithm, i.e., HSMU-SpGEMM, for SpGEMM on modern GPUs. Unlike the traditional hash-based accumulators,

*Corresponding author (luohuizhang@hnu.edu.cn).

¹HSMU-SpGEMM is available at <https://github.com/wuminq/HSMU-SpGEMM>

²GPU shared memory utilization is defined as the ratio of the actual shared memory used during the execution of kernel functions to the total shared memory allocated.

each HSMU-SpGEMM accumulator kernel maintains a sorted column array of length with the maximum NNZ of its allocated rows, so that the hash collisions can be avoided by indexing the sorted array. We further developed different symbolic stages for both small-scale and large-scale matrices to support HSMU-SpGEMM's accumulator. Our evaluation demonstrates that HSMU-SpGEMM optimizes GPU shared memory while maintaining a low hash collision rate for high-performance SpGEMM.

Our main contributions are:

- We systematically analyze the drawbacks of traditional hash-based accumulators, and show the motivation for a high shared memory utilization and collision-free accumulator.
- We propose and design the novel HSMU-SpGEMM accumulator that overcomes the limitations of existing hash-based accumulators, and combines the proposed accumulator with a dense accumulator for a wider SpGEMM support.
- We propose two novel symbolic stages, catering to both small-scale and large-scale matrices, to complement the requirements of HSMU-SpGEMM's new accumulator.

We test HSMU-SpGEMM on more than 300 sparse square matrices from the *SuiteSparse Matrix Collection* [22] on three modern GPUs. Compared with four state-of-the-art hash-based libraries (*Nsparse*, *spECK*, *OpSparse*, and *cuSPARSE*), HSMU-SpGEMM demonstrates the following performance improvements across all 338 sparse matrices: 1) On NVIDIA GeForce RTX 3090 Ti, HSMU-SpGEMM achieves geometric mean speedup of 3.19 (up to 131.64), 1.28 (up to 4.68), 2.34 (up to 107.70), and 5.06 (up to 14.15), respectively; 2) On NVIDIA GeForce RTX 4080 SUPER, HSMU-SpGEMM achieves geometric mean speedup of 3.08 (up to 125.25), 1.24 (up to 4.58), 2.24 (up to 105.74), and 6.92 (up to 18.03), respectively; 3) On NVIDIA GeForce RTX 1650 SUPER, HSMU-SpGEMM achieves geometric mean speedup of 1.41 (up to 70.14), 1.10 (up to 2.05), 1.46 (up to 11.09), and 1.73 (up to 6.02), respectively.

II. BACKGROUND

A. Gustavson's Algorithm for SpGEMM

As shown in Algorithm 1, Gustavson's Algorithm exhibits inherent parallelism as it independently computes each row of matrix C . In the numeric stage, each NZ element a_{ij} in matrix A is associated with a row b_{j*} in matrix B . Subsequently, it is multiplied with each NZ element b_{jk} in the corresponding row to generate a substantial volume of intermediate products. The column indices of these intermediate products correspond to the column indices of the NZ elements in matrix B . The accumulation of intermediate products at specific positions of matrix C is achieved by using the accumulator. The details of the accumulator are provided in the next subsection.

B. SpGEMM Accumulator Design

Accumulators can be categorized into two types: dense and sparse accumulators, according to the storage structure

Algorithm 1 A pseudocode for implementing SpGEMM using Gustavson's row-row algorithm designed for $C = A * B$

```

1: Symbolic stage: Determine the size and allocate space for
   matrix  $C$ 
2: Numeric stage:
3: for each row  $a_{i*}$  in  $A$  do
4:   for each element  $a_{ij}$  in  $a_{i*}$  do
5:     for each element  $b_{jk}$  in row  $b_{j*}$  of  $B$  do
6:       intermediate_product  $\leftarrow a_{ij} \times b_{jk}$ 
7:        $C_{ik} \mathrel{+}= \text{intermediate\_product}$  {Accumulator to
       implement the corresponding addition}
8:     end for
9:   end for
10: end for

```

of the intermediate products [23]. Dense accumulators utilize dense arrays for the accumulation of intermediate products. The widely adopted dense accumulator initially proposed by Gustavson [8], generally consists of three vectors. The first vector stores real values, the second one functions as a tag array to verify the insertion of column indices, and the third one stores the column indices. Dense accumulators exhibit high efficiency for very dense rows but demonstrate poor performance on sparse rows, primarily due to their substantial memory demands. With regard to sparse accumulators, they can be classified into three categories depending on the accumulation methods: merge-based accumulators, ESC-based accumulators, and hash-based accumulators [12].

Merge-based sparse accumulator is implemented in libraries such as *RMerge* [15] and *bhSPARSE* [16]. These accumulators perform multiple iterations, where each iteration vertically merges one NZ element into the final sparse vector. Due to the use of equally-sized temporary arrays during merging, merge-based sparse accumulator exhibits low memory utilization for matrices with significant variations in their density.

ESC (Expand, Sort, and Compress) based accumulators store all intermediate products in temporary memory, followed by sorting based on (row and) column indices and then perform accumulation for values with colliding indices. This approach can ensure efficient load balancing and automatic optimization of memory access patterns, as it operates on all intermediate products independently of their origin. For instance, *CUSP* [17], *bhSPARSE* [16], and *AC-SpGEMM* [18] libraries implement the ESC method. However, ESC method falls short when handling matrices that generate a substantial volume of intermediate products. The inefficiency arises from significant space overhead and time costs due to storing and sorting a considerable number of intermediate products.

Hash-based accumulators utilize hash tables for accumulating intermediate products, leading to a more efficient utilization of memory space, as compared to the above mentioned two sparse accumulators. Hash-based accumulators are great candidates for matrices with moderate length of rows that can be accommodated within the GPU shared memory. *BalancedHash* [24], *Nsparse* [10], *spECK* [12], *OpSparse* [13],

and *cuSPARSE* [19], [20] are well-known libraries that employ hash-based accumulators. *BalancedHash* uses a fixed-size hash table strategy, in which each table processes the same number of intermediate products. Hash collisions are handled by placing the conflicting entries into a spillover queue for reprocessing. This approach requires additional space to maintain work lists, and the space increases with the growing intermediate products (often a large quantity). For other hash-based libraries (*Nsparse*, *spECK*, *OpSparse*, and *cuSPARSE*), computations are performed on a row-by-row basis. When the processed matrix rows are excessively long, hash tables can no longer be accommodated in the limited GPU shared memory and are instead stored in the larger global memory, incurring significant access latency than the shared memory. To address this issue, *spECK* [12] library employs a hybrid approach, combining hash-based and dense accumulators, and dynamically selects the appropriate accumulator based on the density of rows being processed.

A comprehensive overview of the hash-based accumulators and their limitations are presented in Section III.

C. Fundamentals of GPUs

We use NVIDIA GPU architecture and the CUDA programming model as the vehicle to introduce GPUs. Modern GPUs architecture is composed of multiple parallel Streaming Multiprocessors (SMs). Each SM has its CUDA cores and shared memory. Every CUDA core can execute only one thread at a time, while a warp consists of 32 threads. Warps are then grouped into thread blocks, and a collection of thread blocks forms a kernel, referred to as a grid. GPUs also have a global memory (in GBs) accessible to all SMs, albeit with higher latency. Additionally, all SMs can access a shared L2 cache, while each SM has its own local L1-cache-like shared memory for its CUDA cores. The right part of Fig. 1 depicts a simplified GPU architecture, exemplifying the NVIDIA GeForce RTX 3090 Ti. Each SM is capable of executing multiple thread blocks simultaneously, subject to hardware resources such as the number of available CUDA cores and the capacity of shared memory.

D. Shared Memories in GPUs

Shared memories are the fastest memory type in GPUs, in addition to register files. The primary advantage of shared memory is that it allows multiple threads to share data, which positions shared memory as a critical GPU component for efficient parallel computing. Shared memory is accessible to all threads within a single thread block, and it can be viewed as a programmable cache or scratchpad [25], [26], where the programmer explicitly places frequently accessed data. However, the capacity of shared memory is limited. On NVIDIA GeForce RTX 3090 Ti, as shown in the right part of Fig. 1, each SM only has 128KB of shared memory. Therefore, maximizing the utilization of the small yet high-performance shared memory is crucial for modern GPUs.

III. MOTIVATION OF THE WORK

In this section, we first present an overall introduction of existing hash-based accumulators and their limitations. Then, we show the motivation of our new accumulator design that has high shared memory utilization and zero hash collision.

A. Principles of Hash-based Accumulators

Hash-based accumulators first allocate memory space for the hash tables and use the column indices of intermediate products (denoted as *colIp*) as the keys. Then, intermediate products sharing the same column index and key can be efficiently accumulated by mapping them to corresponding positions in the hash table. Fig. 1 shows an overview of where and how hash-based accumulators work with the framework of a modern GPU. As shown in the figure, the hash-based accumulator is maintained in the shared memory. There are several hash tables within each SM. Specifically, when rows of matrix *C* are allocated to SMs in batch, the corresponding hash tables are generated. We further show how hash tables are maintained with an example in Fig. 1. The hash table length is set to 4, with the key table initially set to $\{-1\}$ and the value table initialized as $\{0\}$. A simple hash function, $H(colIp) = colIp \text{ MOD } 4$, is applied. The *key* table ultimately stores the column index of NZ elements of matrix *C*, while their corresponding value is stored in *val* table. After shrinking the hash table and sorting it in ascending order according to the column indices, we obtain the final *Ccol* and *Cval* array of matrix *C*.

For traditional hash-based accumulators, hash collisions are inevitable when intermediate products with different column indices are mapped to the same location in the hash table, even if the length of the hash table is increased (e.g., *OpSparse*). Existing hash-based accumulator libraries present an inherent trade-off between hash collision rate and shared memory utilization. To better understand the relationship between the number of hash collisions and shared memory utilization, and their impact on performance, we benchmark several SpGEMM libraries with the same matrix *cant* on NVIDIA 3090 Ti, and the results are presented in Table I³.

TABLE I: Comparison of different hash-based libraries on the number of hash collisions in the numeric stage (Collisions), ratio of Collisions to the number of intermediate products (Ratio), length of the hash table (Length), shared memory utilization (Utilization), and end-to-end runtime on NVIDIA 3090 Ti.

Library	Collisions	Ratio	Length	Utilization	Runtime
Nsparse	1078031566	400.03%	$1 \times \text{max NNZ}$	70.67%	11.30 ms
OpSparse	7410714	2.75%	$2 \times \text{max NNZ}$	35.35%	9.02 ms
Ours	0	0%	$1 \times \text{max NNZ}$	71.69%	7.80 ms

It is worth noting that the Ratio of *Nsparse* exceeds 400%, which means that its accumulator has to access the hash

³*spECK* is not included in the comparison due to its layer-by-layer code encapsulation, which makes it difficult to measure the number of hash collisions.

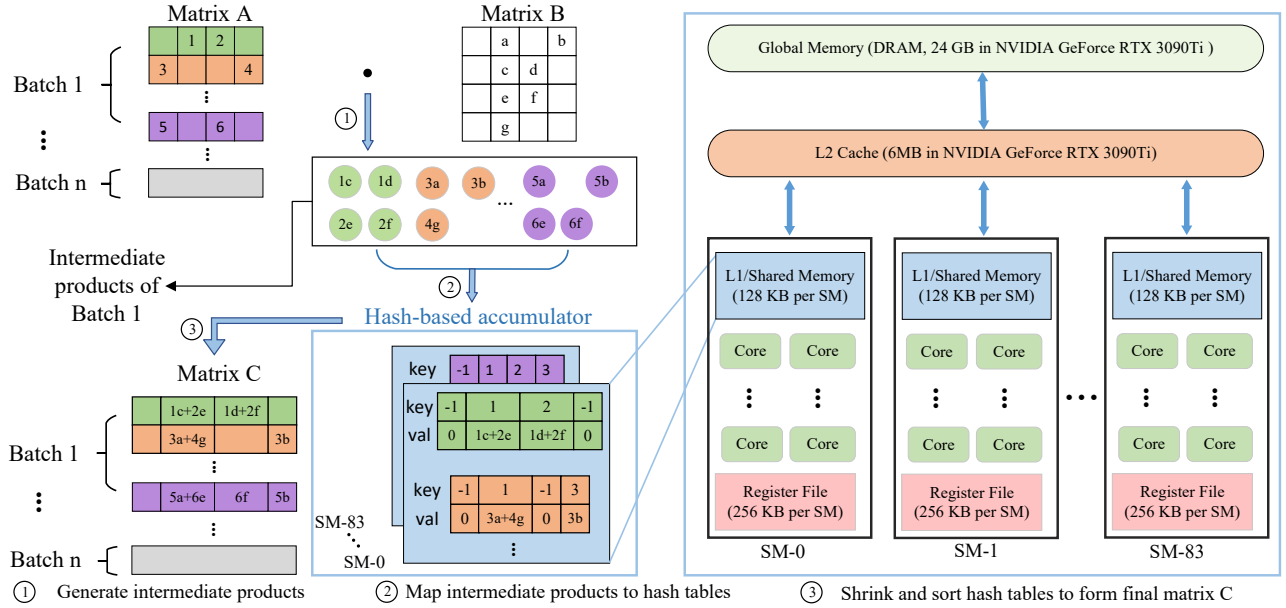


Fig. 1: The left side of the figure depicts the fundamental process of hash-based SpGEMM on a GPU. Batch processing is often used to compute the final matrix C . The simplified GPU architecture, as depicted on the right side, is demonstrated using NVIDIA GeForce RTX 3090 Ti's specs as a representative example.

table four times on average before the right position of an intermediate product is found. As for *OpSparse*, its hash collisions are much smaller than *Nsparse*, while the shared memory utilization is only 35.35%. Although prior works have extensively explored the trade-off between shared memory utilization and hash collisions, our new accumulator simultaneously optimize the collision ratio and the memory utilization, leading to superior end to end program performance measured on modern GPUs (as shown in Table I). Our novel accumulator design breaks the inherent limitation of existing hash-based methods, while ensures exceptional performance and effectively utilizing shared memory resources.

B. Philosophy of HSMU-SpGEMM Accumulator Design

The core function of the hash-based accumulators is the positioning of intermediate products on the hash table, where in conventional methods, the positioning process relies solely on the column indices of the intermediate products, as illustrated in the left part of Fig. 2. While in our new accumulator design, we re-design the positioning process by incorporating an array that keeps track of the sorted column indices of NZ elements in matrix C . As illustrated in the right part of Fig. 2, our new accumulator employs the function *findInSorted()* for positioning. This function takes the column indices of the intermediate products and a pre-sorted array of column indices for NZ elements as its input, and generates the corresponding positions. Doing this eliminates the hash collisions found in traditional hash-based methods, thus obviating the need to increase the length of the value array to avoid collisions and unnecessary resource consumption. Note that the pre-

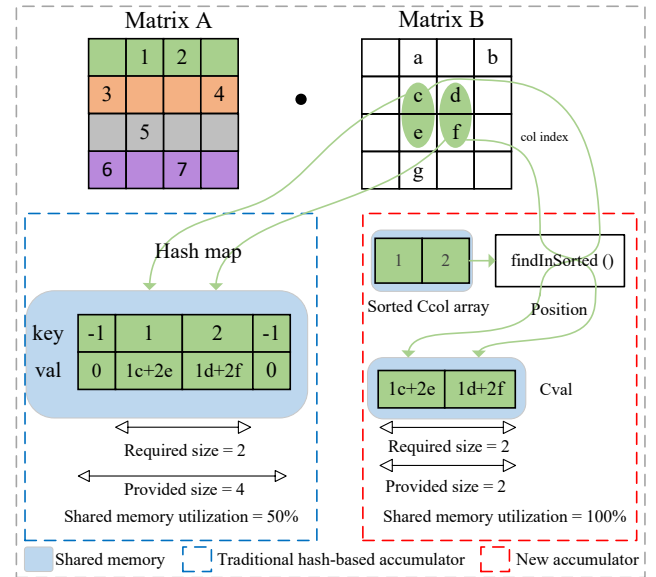


Fig. 2: The comparison of traditional hash-based accumulator (*OpSparse* as an example) and HSMU-SpGEMM accumulator. Taking the multiplication of two 4×4 matrices A and B to produce the first row of matrix C as an illustrative example. The figure presents the fundamental process of our new accumulator and provides a comparison of its shared memory utilization with that of the traditional hash-based accumulator.

sorted array is computed during the symbolic stage of HSMU-SpGEMM, which is prior to the accumulator computation.

Motivational Example As shown in Figure 2, traditional hash-based accumulator allocates a hash table of length 4, while the intermediate products of matrix C require a table of actual length of 2. Hence, the shared memory utilization in this case is 50%, which does not guarantee hash collision free. In contrast, our new accumulator achieves 100% shared memory utilization by only allocating a sorted column array and a value array with a length 2. Our approach further eliminates hash collisions through the function of *findInSorted()*. Note that at numerical stage, $Ccol$ and $Cval$ in our HSMU-SpGEMM accumulator are ready to use for the final matrix C while the traditional hash-based accumulators have to sort the *key* and *val* tables to obtain $Ccol$ and $Cval$. We discuss the implementation details of HSMU-SpGEMM in Section IV.

IV. HSMU-SpGEMM

This section first describes the implementation of our HSMU-SpGEMM accumulator. Specifically, we analyze the advantages of replacing hash lookup with binary search in function *findInSorted()*. Then, Sections IV-B and IV-C discuss how to obtain the pre-sorted $Ccol$ array for our new accumulator. Finally, we provide a workflow chart for HSMU-SpGEMM in Section IV-D.

A. HSMU-SpGEMM Accumulator Design

Algorithm 2 describes the design of our new accumulator. The design relies on a pre-sorted $Ccol$ array to compute a row of final matrix C on GPUs. The algorithm maps each row of C to a single GPU thread block. Within a thread block, one warp is allocated to a NZ element of matrix A in a stride of $wnum$, while a warp computes on the corresponding row of B for the allocated NZ elements with a stride of $warpSize$ (Lines 6-14). In particular, each thread within a warp handles one NZ element of B , i.e., the thread computes the intermediate product ($aval * Bval[k]$) and accumulates into the $Cval$ array ($shared_val$) at the position (pos) obtained from the function *findInSorted()* (Lines 10-13). Note that pos is also used as the starting point for each search in *findInSorted()*, which reduces the search space as pos progressively increases.

There are two key points of HSMU-SpGEMM accumulator. *First*, generating the sorted $Ccol$ array requires auxiliary structures that are maintained in the symbolic stage (discussed in the next subsection). *Second*, designing the function *findInSorted()*. Our *findInSorted()* is built upon a binary search algorithm, which is a low overhead, memory-efficient, and stable implementation. We make the design choice based on the following observations when compared to the original hash table based solutions: 1) The biggest advantage of hashing over binary search is that it is much cheaper to add or remove an item from a hash table. However, in HSMU-SpGEMM, sorted arrays are pre-determined in the symbolic stage, so our new accumulator does not need to change the sorted array in the numeric stage. Hence, the shortcoming of maintaining a sorted structure for binary search does not exist;

Algorithm 2 HSMU-SpGEMM accumulator implementation. We take the i -th row of matrix C as an example.

```

1: Load pre-sorted Ccol array into shared_col
2: Initialize shared_val to {0};
3:  $tid \leftarrow threadIdx \% warpsize$ ;
4:  $wid \leftarrow threadIdx / warpsize$ ;
5:  $wnum \leftarrow blockDim / warpsize$ ;
6: for  $j = Aptr[i] + wid$  to  $Aptr[i + 1]$ , step =  $wnum$ 
   do
7:    $acol \leftarrow Acol[j]$ ;
8:    $aval \leftarrow Aval[j]$ ;
9:    $pos \leftarrow 0$ ;
10:  for  $k = Bptr[acol] + tid$  to  $Bptr[acol + 1]$ , step =
     $warpSize$  do
11:     $pos \leftarrow findInSorted(shared\_col, pos, Bcol[k]);$ 
12:     $atomicAdd(shared\_val + pos, aval * Bval[k]);$ 
13:  end for
14: end for

```

2) One disadvantage of the hash table is that when a collision occurs, it will affect other hash positions and may cause chain collisions [27], leading to low hashing performance. While for binary search, its performance is stable, and the worst case is $O(\log N)$. In this case, binary search outperforms the hash lookup method; 3) Binary search works better for complex operations such as range queries. In this scenario, each thread gradually narrows down the search range on the *shared_col* array by continuously updating the variable pos , thereby reducing the number of lookups to a certain extent; Nevertheless, the worst-case time complexity of the binary search for dense and large data is $O(\log N)$, while the ideal time of hash table is $O(1)$. The increase in time complexity may potentially degrade the search performance of our accumulator design in certain cases. To mitigate this issue, HSMU-SpGEMM augments our binary search accumulator with another dense and efficient accumulator, chosen from *spECK* [12]. The choice between binary search accumulator and the dense accumulator can be tuned by a threshold parameter which will be discussed in Section V-B.

B. Generating the Sorted $Ccol$ Array

Two auxiliary mask matrices, named mask B and mask C , are introduced to calculate the sorted $Ccol$ array. The mask B and the mask C are with the same data structure, as shown in Fig. 3. The process of generating the sorted $Ccol$ array involves three steps: 1) generation of the mask B based on matrix B ; 2) generation of the mask C based on matrix A and the mask B ; 3) extraction of information regarding NZ elements of matrix C from the mask C . Fig. 3 illustrates these three steps using two 4x4 matrices as an example. For the first step, as depicted in the upper part of the figure, the mask B is generated based on the element values in matrix B . We mark a value of 1 in the corresponding column index position for each row containing NZ elements, and 0 otherwise. Notably, each 0 or 1 value is stored as a single bit. 64 bits are used as

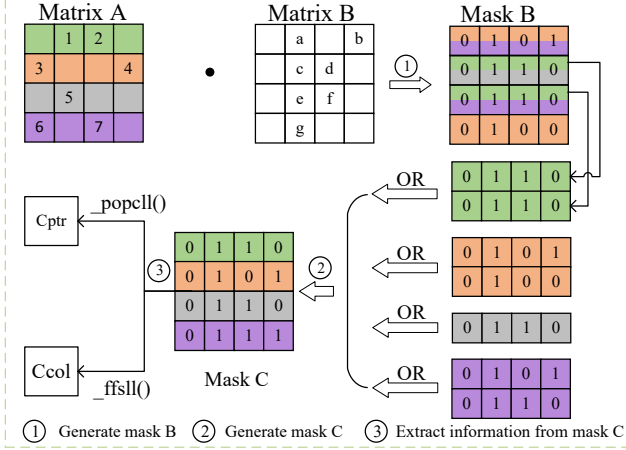


Fig. 3: Generating the sorted $Ccol$ array for small-scale matrices.

the basic unit to store auxiliary mask matrices, and this choice is made to reduce the overhead in subsequent processing.

For the second step, the lower-right part of the figure illustrates the process of creating the mask C . Each row of matrix A corresponds to one or multiple rows of the mask B , as different color coding indicates. The color coding is based on the mapping relationship between a single row of matrix A and multiple rows of matrix B (and mask B) as described in Gustavson's Algorithm II-A. For each row of A , we perform a direct bitwise OR operation with all corresponding rows of the mask B , resulting in the corresponding row of the mask C . Since the process for each row of A is independent, it can be embarrassingly parallelized.

For the third step, the number and indices of 1 in mask C provide information ($Cptr$ and $Ccol$) for the $Cval$ array. Library functions `__popc()` and `__ffsll()` are utilized to extract the above information. `__popc()` can count the number of NZ bits in a 64-bit integer, and generate the $Cptr$ array in CSR format [28]. `__ffsll()` is used to find the position of the first set bit (with a value of 1) in a 64-bit integer starting from the least significant bit. By calling `__ffsll()` multiple times, we obtain the column indices of all NZ elements, which are then sorted to obtain the sorted $Ccol$ array.

C. Enhanced Version of Generating the Sorted $Ccol$ Array

As matrix size increases, the sparsity results in a small proportion of valid 1 entries in the mask matrix, making space inefficient. For example, when the number of columns of matrix B exceeds 2048, a warp will not be able to process one row-wise OR operation at a time, where each thread handles only one 64-bit unit at a time, causing performance degradation. To deal with this issue, the compressed mask matrix format similar to CSR is introduced for large-scale matrices.

Fig. 4 is used as an example to demonstrate the compressed mask matrix format. We devise a specialized B with dimensions of 4 rows and 192 columns. Its CSR storage format is depicted in the upper-left corner of the figure. Specifically, for

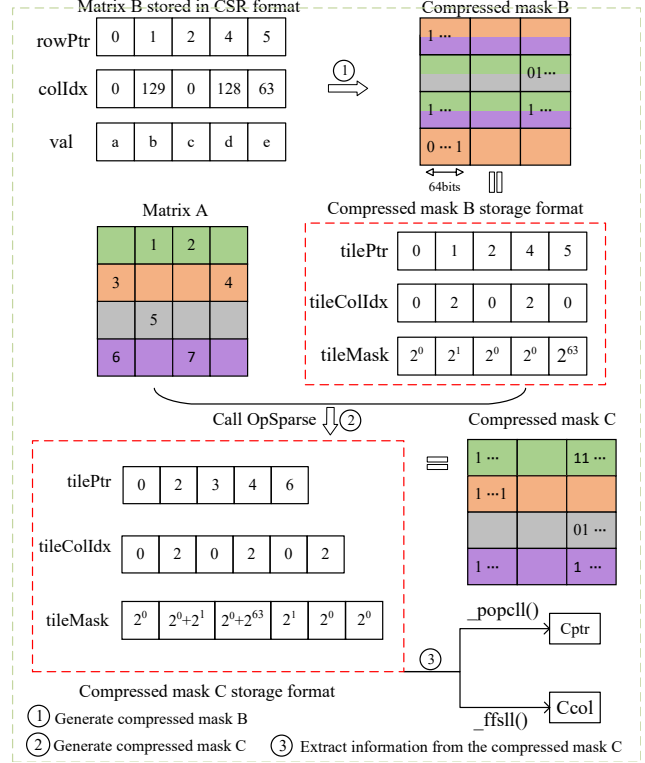


Fig. 4: Generating the sorted $Ccol$ array for large-scale matrices. Each row of A corresponds to one or multiple rows of the compressed mask B , and this correspondence is visually represented through color coding. Upon calling the *OpSparse*, matrix A and the compressed mask B combine to yield the compressed mask C , effectively storing information about the NZ elements in final matrix C .

step 1): every 64 bits of mask B is treated as a basic unit. Thus, the compressed mask B can be regarded as a smaller matrix comprising 4 rows and 3 columns. The compressed mask B is represented using three arrays: $tilePtr$, $tileCol$, and $tileMask$. The $tilePtr$ array stores the indices of the starting positions of the first NZ units in each compressed mask row of B , the $tileCol$ array stores the column indices of the NZ units, and the $tileMask$ array stores the bit patterns of the NZ elements. It should be noted that elements within the $tileMask$ array are stored as 64-bit data types.

For step 2): *OpSparse* is chosen due to its excellent interface design and impressive performance. The symbolic stage of the *OpSparse* library is employed to determine the number of NZ units in the compressed mask C and generate the $tilePtr$ array of compressed mask C . Furthermore, we modify the numeric stage of the *OpSparse* to replace standard multiplication and accumulation operations with bitwise OR operations on the units of compressed mask B . This modification aims to generate the $tileCol$ and $tileMask$ arrays effectively.

For step 3): it follows the same method as described in Section IV-B.

D. Flowchart of HSMU-SpGEMM

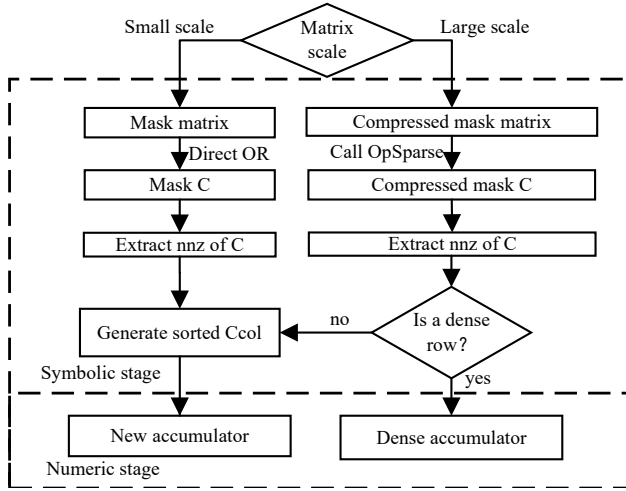


Fig. 5: Workflow of HSMU-SpGEMM. The extraction process for column indices of NZ elements in the non-dense rows of matrix C is arranged during the symbolic stage, deviating from conventional methods.

Based on the discussion thus far, Fig. 5 presents a complete flowchart of HSMU-SpGEMM. HSMU-SpGEMM differs from traditional hash-based libraries in both the symbolic and numerical stages. For conventional hash-based symbolic stages, they determine the NNZ of final matrix C by mapping intermediate products onto a hash table through a hash function. For HSMU-SpGEMM symbolic stage, by performing logical OR operations, HSMU-SpGEMM obtains the auxiliary mask structure containing positional information for the NZ elements of final matrix C . Subsequently, HSMU-SpGEMM extracts NNZ and the sorted column index array of matrix C from the auxiliary mask structure. Section V-E will discuss the overheads associated with generating the auxiliary mask structure and extracting the sorted column index array. For conventional hash-based numeric stages, *OpSparse*, *Nsparse*, and *cuSPARSE* only use hash accumulators, *spECK* uses a combination of hash accumulator and dense accumulator. For HSMU-SpGEMM numeric stage, HSMU-SpGEMM uses a combination of the new accumulator and the dense accumulator. Section V-B will discuss how to choose between these two accumulators and the advantages of combining them.

Similar to existing libraries, HSMU-SpGEMM processes the entire SpGEMM using multiple rows as a batch, as illustrated in Fig. 1. For load balancing strategy, HSMU-SpGEMM reuses *Nsparse*'s binning strategy [10], where rows are allocated into corresponding bins based on their NNZs, and the number of bins is determined by the hardware.

V. EVALUATION

A. Experiment Setup

Hardware Platform We evaluate HSMU-SpGEMM on three modern GPUs with different peak performance: NVIDIA GeForce RTX 3090 Ti and NVIDIA GeForce RTX 4080

SUPER for higher performance (peak performance is 625.0 GFLOPS and 816.0 GFLOPS for FP64/double precision, respectively) and NVIDIA GeForce RTX 1650 SUPER for lower performance (peak performance is 138.0 GFLOPS for FP64/double precision). These GPUs also have different architectures, i.e., Ampere, Ada Lovelace, and Turing, respectively. The CUDA and GPU driver versions are all 11.4 and 525.105.17, respectively. The source code is compiled with the -O3 optimization level. The operating systems are Ubuntu 22.04. The CPU paired with the 3090 Ti is the 12th Gen Intel(R) Core(TM) i9-12900K, the CPU paired with the 4080 SUPER is the Intel(R) Core(TM) i9-14900K, while the CPU paired with the 1650 SUPER is the 11th Gen Intel(R) Core(TM) i5-11400F. We compare our library with four state-of-the-art hash-based SpGEMM libraries (using their respective recommended configurations): *Nsparse*, *spECK*, *OpSparse*, and *cuSPARSE* v11.4 (*cusparseSpGEMM_compute* [29] function). There are also methods employing hardware acceleration [30]–[33], which are executed on simulators. This work focuses on real accelerators, particularly GPUs. Simulation based comparisons are beyond the scope of this study.

Datasets As for the input matrices, to saturate modern GPUs, we tested all 338 sparse square matrices in the *SuiteSparse Matrix Collection* [22] that have no less than 100 million floating point operations when computing both $C = A^2$ and $C = AA^T$, similar to prior work [34]. We also evaluate 18 representative matrices on NVIDIA 3090 Ti to benchmark the performance on each library. These 18 matrices, are previously developed and used for benchmarking and testing in prior works [12], [13], [18], [34], [35]. Table II shows the detailed information for the 18 matrices (matrix ID from 1 to 18), which exhibit diversity with compression ratios ranging from 1 to 28 and number of rows/columns spanning from 8.3K to 12.0M. Furthermore, we test four extremely large matrices (matrix ID from 19 to 22 in Table II), which are previously evaluated in [13], [34] to verify the effectiveness of combining the new binary search accumulator with the dense accumulator in HSMU-SpGEMM.

B. Parameter Setting

This section discusses how to choose between our new binary search accumulator and the dense accumulator for HSMU-SpGEMM, and illustrates the advantages of combining these two types of accumulators.

We consider NNZ per row in matrix C as a criterion for selecting an accumulator, since the NNZ per row primarily impacts the number of searches for HSMU-SpGEMM's new binary search accumulator. To study the impact of different NNZ thresholds on performance, we vary the NNZ thresholds from 128 to 24576, and plot the geometric mean of GFLOPS from all 338 matrix when running HSMU-spGEMM on 3090 Ti in Fig. 6. We observe that when the NNZ threshold is set to a low value (128), the geometric mean of the 338 matrices is low (16.41 GFLOPS). This is because our new accumulator only handles rows in matrix C with NNZ less

TABLE II: Information of 22 matrices, including 18 representative matrices and 4 extremely large matrices. The **np(C)** represents the number of intermediate products when computing $C = A * A$. The **Comp.** represents the ratio between the number of intermediate products and the $\text{nnz}(C)$.

ID	Matrix A	Rows(A)	nnz(A)	np(C)	nnz(C)	Comp.
1	cant	62.5K	4.0M	269.5M	17.4M	15
2	pwtk	217.9K	11.6M	626.1M	32.8M	19
3	offshore	259.8 K	4.2M	71.3M	23.4M	3
4	cage12	130.2K	2.0M	34.6M	15.2M	2
5	scircuit	171.0K	1.0M	8.7M	5.2M	1
6	wiki-Vote	8.3K	0.1M	4.5M	1.8M	2
7	patents_main	240.5K	0.6M	2.6M	2.3M	1
8	poisson3Da	13.5K	0.4M	11.8M	3.0M	3
9	pd1HYS	36.4K	4.3M	555.3M	19.6M	28
10	rma10	46.8K	2.4M	156.5M	7.9M	19
11	shipsec1	140.9K	7.8M	450.6M	24.1M	18
12	consph	83.3K	6.0M	463.8M	26.5M	17
13	conf5_4-8x8-05	49.2K	1.9M	74.8M	10.9M	6
14	filter3D	106.4K	2.7M	86.0M	20.2M	4
15	mac_econ_fwd500	206.5K	1.3M	7.6M	6.7M	1
16	asia_osc	12.0M	25.4M	56.9M	42.8M	1
17	af_shell10	1.5M	52.7M	1.8B	142.7M	12
18	hood	220.5K	10.8M	562.0M	34.2M	16
19	case39	40.2K	1.0M	4.0B	404.7M	10
20	gupta3	16.8K	9.3M	30.7B	270.9M	113
21	TSOPF_FS_b300_c2	56.8K	8.8M	53.9B	805.7M	66
22	delaunay_n24	16.8M	100.7M	633.9M	347.3M	1

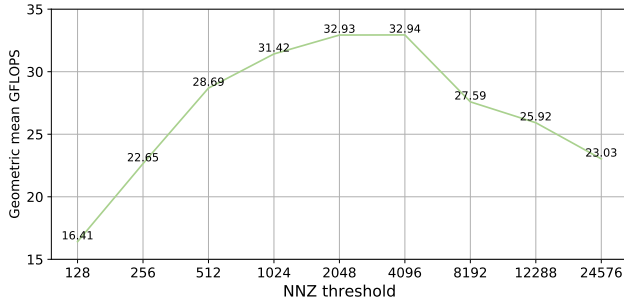


Fig. 6: Experiments on different selection thresholds for our new binary search accumulator and dense accumulator.

than 128, while the dense accumulator handles all rows with NNZ greater than 128. Consequently, many rows that are not dense enough are still processed by the dense accumulator, leading to poor performance. Another extreme scenario occurs when the NNZ threshold is set very high (24576), resulting in lower performance (23.03 GFLOPS). This is due to many dense rows being assigned to our new accumulator, leading to an excessive number of searches and consequently degraded performance. We select 4096 as the threshold for choosing our new accumulator and dense accumulator, as HSMU-SpGEMM achieves maximum performance at this value.

To better demonstrate the advantage of combining the two accumulators in HSMU-SpGEMM, we present the experimental results (shown in Fig. 7) on four extremely large scale matrices. For libraries that only use hash accumulators, many dense rows will only result in slow global memory processing,

leading to poor performance or failure to complete the calculation due to out-of-memory errors. Compared to *spECK*, which also combines dense accumulator, HSMU-SpGEMM demonstrates better performance. This is attributed to the higher shared memory utilization of our new accumulator compared to the hash accumulator, resulting in fewer batches generated during SpGEMM execution, as shown in Fig. 1.

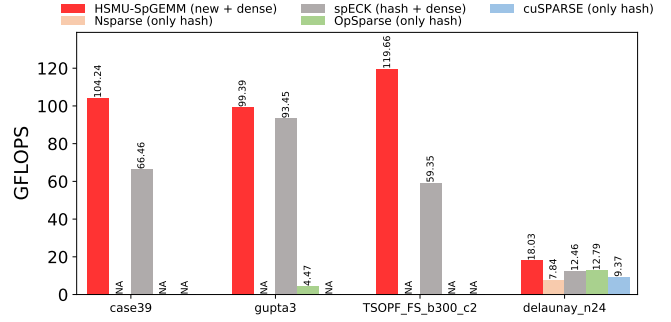
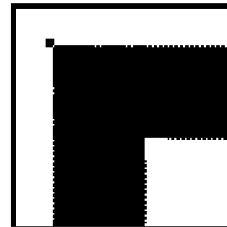


Fig. 7: Performance comparison on four extremely large matrices. NA means that the corresponding library fails to complete the computation.

C. Overall Performance

As shown in Table III, overall HSMU-SpGEMM achieves highest GFLOPS across all 338 matrices on all three evaluated GPUs. Specifically, compared to *Nsparse*, *spECK*, *OpSparse*, and *cuSPARSE*, on 3090 Ti, HSMU-SpGEMM achieves geometric mean speedup of 3.19, 1.28, 2.34, and 5.06, respectively; On 4080 SUPER, HSMU-SpGEMM achieves geometric mean speedup of 3.08, 1.24, 2.24, and 6.92, respectively; On 1650 SUPER, HSMU-SpGEMM achieves geometric mean speedup of 1.41, 1.10, 1.46, and 1.73, respectively. For the results obtained on the 1650 SUPER, limited hardware resources mitigate the impact of the inherent algorithmic characteristics on performance. Consequently, the performance differences among the libraries gradually diminish. The geometric mean GFLOPS for each library are as follows: 6.52 (HSMU-SpGEMM), 4.61 (*Nsparse*), 5.93 (*spECK*), 4.46 (*OpSparse*), and 3.77 (*cuSPARSE*).



(a) Max speed up matrix



(b) Min speedup matrix

Fig. 8: NZ element patterns of two kinds of matrices. The black regions represent non-zero elements at corresponding positions.

We further investigate the impact of a matrix's NZ pattern on the performance of HSMU-SpGEMM. In general, we

TABLE III: Overall performance comparison among the five libraries based on 338 matrices. The information in the table includes: 1) Speedup of HSMU-SpGEMM over the other four libraries, we present three kinds of speedups, i.e., minimum, maximum, and geometric mean; 2) Geometric mean GFLOPS for each library;

Platform	Library	Speedup of HSMU-SpGEMM			Mean GFLOPS
		Min	Max	Geometric mean	
3090 Ti	Nsparse	0.18	131.64	3.19	10.33
	spECK	0.17	4.68	1.28	25.83
	OpSparse	0.15	107.70	2.34	14.10
	cuSPARSE	0.41	14.15	5.06	6.51
	Ours	-	-	-	32.94
4080 SUPER	Nsparse	0.18	125.25	3.08	13.44
	spECK	0.16	4.58	1.24	33.35
	OpSparse	0.14	105.74	2.24	18.45
	cuSPARSE	0.54	18.03	6.92	5.98
	Ours	-	-	-	41.39
1650 SUPER	Nsparse	0.18	70.14	1.41	4.61
	spECK	0.16	2.05	1.10	5.93
	OpSparse	0.4	11.09	1.46	4.46
	cuSPARSE	0.26	6.02	1.73	3.77
	Ours	-	-	-	6.52

TABLE IV: Proportion of matrices for each library achieving from best to worst performance among all the five libraries.

Platform	Library	Best	2nd best	3rd best	4th best	Worst
3090 Ti	Nsparse	3.55%	11.54%	31.07%	29.59%	27.22%
	spECK	12.13%	24.56%	26.04%	37.87%	0.59%
	OpSparse	20.41%	36.98%	16.86%	23.37%	1.78%
	cuSPARSE	1.48%	1.48%	15.98%	7.40%	70.12%
	Ours	62.43%	25.44%	10.06%	1.78%	0.30%
4080 SUPER	Nsparse	1.78%	12.43%	33.43%	32.25%	25.16%
	spECK	14.20%	26.04%	26.63%	33.14%	0.59%
	OpSparse	20.71%	36.39%	16.27%	24.56%	1.48%
	cuSPARSE	0.30%	0.59%	15.38%	6.21%	71.89%
	Ours	63.02%	24.85%	8.28%	3.85%	0.88%
1650 SUPER	Nsparse	7.72%	23.83%	31.54%	17.11%	19.80%
	spECK	21.14%	28.86%	27.85%	20.81%	1.34%
	OpSparse	17.45%	21.48%	2.68%	2.35%	56.04%
	cuSPARSE	1.01%	1.01%	10.40%	6.38%	81.21%
	Ours	52.68%	19.13%	10.07%	2.35%	15.77%

found that HSMU-SpGEMM performs the best when the distribution of NZ elements is very compact. For example, HSMU-SpGEMM achieves the maximum speedup of 131.64 (compared to *Nsparse*) and 107.70 (compared to *OpSparse*), particularly on the matrix *vsp_model1_crew1_cr42_south31*. As shown in Fig. 8(a), *vsp_model1_crew1_cr42_south31* presents an extremely compact distribution of NZ elements. This compact distribution significantly increases the likelihood of hash collisions in traditional hash-based libraries. The reason is that they use the hash function “hash = (col index * scaling factor) MOD length of the hash table” and linear probing for hash collision. The disadvantage of clustering issues for linear probing can lead to a sharp performance degradation. Interestingly, this scenario is particularly well-suited for HSMU-SpGEMM. This is because: 1) The implementation

process of our new accumulator in HSMU-SpGEMM is not affected by whether the NZ elements are compact. In our new accumulator, intermediate products with different column indices do not result in any collisions when mapped into the *Cval* array; 2) The compact NZ structure in HSMU-SpGEMM results in reduced overhead during the symbolic stage, requiring fewer 64-bit basic units and performing fewer OR operations in the compressed mask structure.

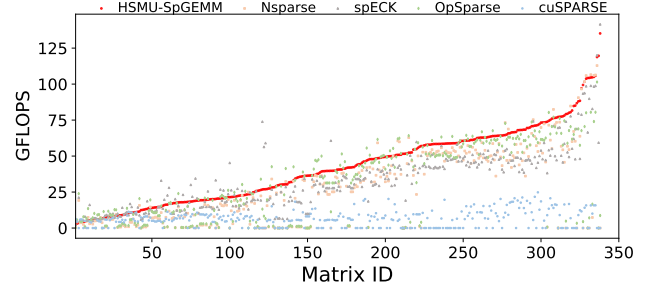


Fig. 9: Scatter plots of performance of all 338 matrices on 3090 Ti. Due to space reasons, the results of the other two platforms are not shown.

On the other hand, we observe that HSMU-SpGEMM achieves the min speedup compared to *Nsparse*, *OpSparse*, and *spECK* simultaneously on the matrix *in-2004*. Fig. 8(b) shows that the distribution of NZ elements in the matrix *in-2004* is highly scattered. The highly scattered patterns of NZ elements imply a lower collision rate for hash-based libraries (resulting in higher performance). However, these patterns are unfriendly to HSMU-SpGEMM for the following reasons: 1) They result in increased symbolic stage overhead since they require more 64-bit basic units and OR operations; 2) For rows processed with a dense accumulator in HSMU-SpGEMM, highly scattered patterns of NZ elements means more iterations.

To show more details of the effectiveness of HSMU-SpGEMM, we also examine the proportion of matrices for each library achieving from best to worst performance. As shown in Table IV, HSMU-SpGEMM achieves the best (worst) performance among all the libraries for 62.43% (0.30%) of all 338 matrices on 3090 Ti, for 63.02% (0.88%) of the matrices on 4080 SUPER, and for 52.68% (15.77%) of the matrices on 1650 SUPER. To show more performance details, we also show scatter plots of the performance of the five libraries on the 3090 Ti in Fig. 9. The results show that HSMU-SpGEMM works consistently better for majority of the evaluated matrices. We further investigate the reason behind the increase in the proportion of matrices with the worst performance for HSMU-SpGEMM, rising from 0.30% on 3090 Ti to 15.77% on 1650 SUPER. HSMU-SpGEMM experiences a performance decrease due to the necessity of invoking the *OpSparse* library for performing OR operations on the compressed mask structure during the symbolic stage. On 1650 SUPER, the limited capacity of shared memory results in more OR operations on very long rows being forced to move

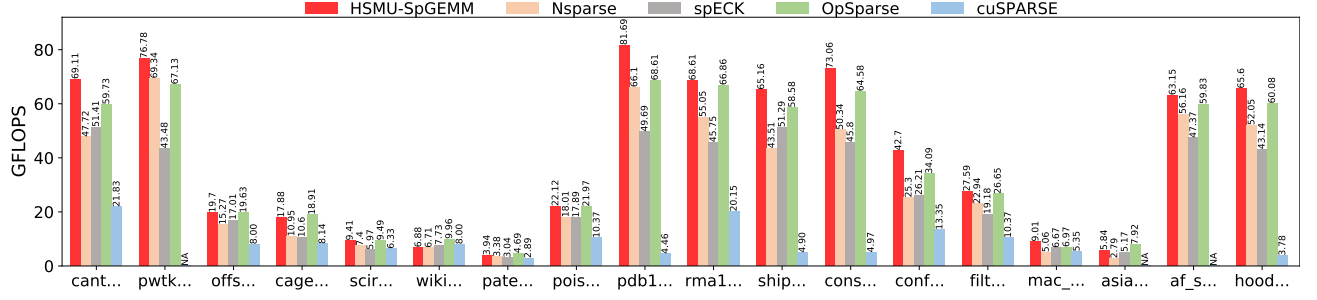


Fig. 10: Comparing overall performance between HSMU-SpGEMM and four hash-based libraries: *OpSparse*, *Nsparse*, *spECK*, *cuSPARSE* v11.4 on 3090 Ti, focusing on 18 representative double-precision matrices from Table II.

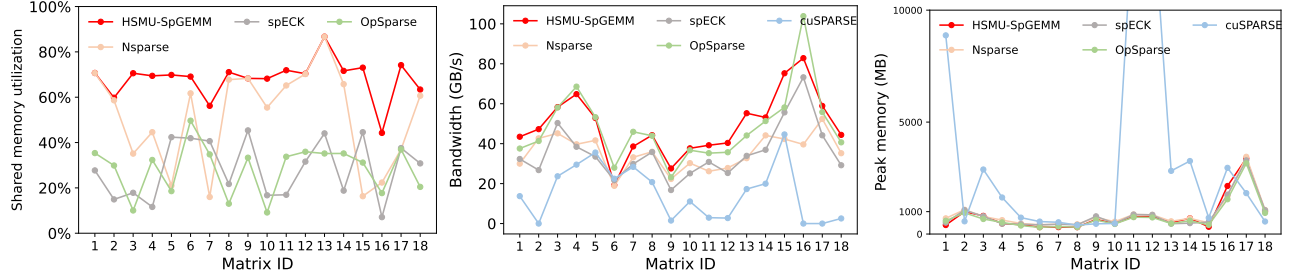


Fig. 11: Evaluation of 18 representative matrices on shared memory utilization (numeric stage), bandwidth, and peak memory on 3090 Ti. The shared memory utilization of *cuSPARSE* library is not measurable because of its closed source code.

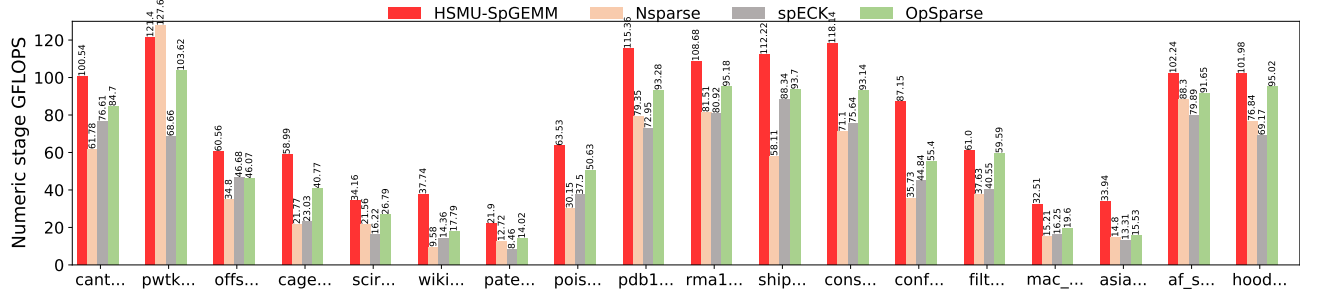


Fig. 12: Performance comparison on the numeric stage for HSMU-SpGEMM, *OpSparse*, *Nsparse*, and *spECK* on 3090 Ti. The numeric stage performance of *cuSPARSE* library is not measurable because of its closed source code.

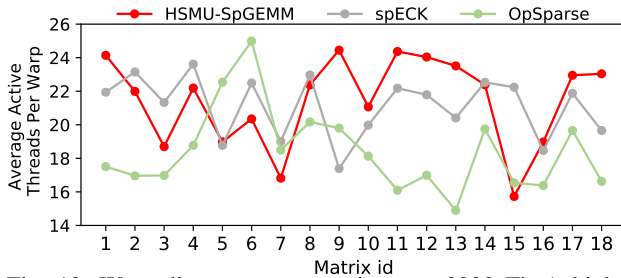


Fig. 13: Warp divergence comparison on 3090 Ti. A higher value indicates less significant warp divergence.

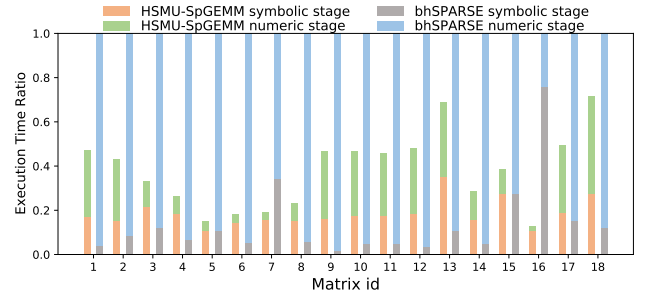


Fig. 14: Time breakdown of HSMU-SpGEMM and *bhSPARSE* on 3090 Ti, normalized to overall time of *bhSPARSE*.

to global memory, causing a certain degree of performance reduction. Nevertheless, HSMU-SpGEMM still achieves the best performance on more than half of the matrices on 1650 SUPER.

D. Detailed Comparison on Representative Matrices

Fig. 10 presents a performance comparison between five libraries on 18 representative matrices. HSMU-SpGEMM

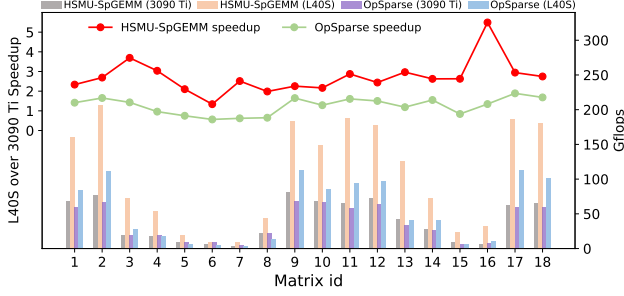


Fig. 15: Performance and speedup of HSMU-SpGEMM and OpSparse on L40S and 3090 Ti platforms.

outperforms *Nsparse*, *spECK*, *OpSparse*, and *cuSPARSE* in 18/18, 17/18, 13/18, 17/18 cases, respectively. To explore more details of each library, we further evaluate the shared memory utilization (numeric stage), bandwidth, and peak memory as shown in Fig. 11. For shared memory utilization, the average utilization of HSMU-SpGEMM reaches 68%, which is higher than the other three libraries, especially the *OpSparse* and *spECK*. The reason is that both *OpSparse* and *spECK* extend the length of the hash table, resulting in a certain waste of resources. Although *spECK* maintains a maximum hash table filling rate of 66%. However, for certain matrices with uniform distributions (e.g., *pwtk*, *cant*, *consph*, etc.), due to the absence of global load balancing, short rows result in the allocation of excessively long hash tables, which leads to inefficient utilization of shared memory for these specific matrices. For bandwidth, HSMU-SpGEMM outperforms other libraries for the majority of matrices. As for peak memory, the results of

TABLE V: Peak memory comparison between HSMU-SpGEMM with *OpSparse* on four extremely large matrices;

Matrix	Number of long rows	Peak memory (GB) Ours	OpSparse
case39	20069	4.94	NA
gupta3	16762	3.61	9.81
TSOPF_FS_b300_c2	28082	9.68	NA
delaunay_n24	0	8.29	6.68

the five libraries are: HSMU-SpGEMM (820.9MB), *Nsparse* (887.1MB), *spECK* (839.2MB), *OpSparse* (763.4MB), and *cuSPARSE* (3507.9MB). *OpSparse* achieves the lowest average peak memory among these 18 matrices, thanks to its strategy of ‘MINIMIZING GLOBAL MEMORY USAGE FOR THE METADATA’ [13]. Note that HSMU-SpGEMM achieves the second lowest peak memory. However, HSMU-SpGEMM will outperform *OpSparse* if matrix *C* is with large number of long rows (i.e., a row with more than 4096 NZ elements is considered a long row and processed in global memory for *OpSparse* on 3090 Ti). As shown in Table V, for matrix *gupta3* (whose matrix *C* contains 16,762 long rows), *OpSparse*’s peak memory reaches 9.81GB due to the necessity of allocating a significant number of rows to global memory. In contrast, HSMU-SpGEMM’s peak memory is only 3.61GB. Moreover, for matrix *case39* and *TSOP_FS_b300_c2*, *OpSparse* fails to

complete the calculation due to an out-of-memory error, while HSMU-SpGEMM could complete the calculation normally.

Fig 12 shows that the performance benefits of HSMU-SpGEMM in the numeric stage follows closely with its high shared memory utilization (Fig 11). While measuring the end-to-end performance (Fig 10), the overhead of HSMU-SpGEMM in symbolic stage is included.

We also evaluate warp divergence of the numeric stage introduced by HSMU-SpGEMM binary search and hash collisions from *spECK* and *OpSparse*. Fig 13 shows the ‘Avg.ActiveThreadsPerWarp’ metric from *Nsight Compute* tool across 18 representative matrices. We can see that the warp divergence of the three methods is similar, with the average active threads per warp around 20.

Fig 14 compares performance breakdown of HSMU-SpGEMM with an existing non-hash based approach, *bhSPARSE* [16]. Across 18 presentative matrices, HSMU-SpGEMM outperforms *bhSPARSE* in overall performance. Note that HSMU-SpGEMM requires more time than *bhSPARSE* for most matrices in symbolic stage as *bhSPARSE* only computes the upper bound of NNZ; while in numeric stage, HSMU-SpGEMM consistently runs faster than *bhSPARSE*.

To test the performance potential of HSMU-SpGEMM on data-center high-performance GPUs, we benchmark HSMU-SpGEMM and *OpSparse* on a NVIDIA L40S (which has a $2.29 \times$ faster FP64 performance than 3090 Ti). Fig 15 (bar chart) shows the significant performance of HSMU-SpGEMM over *OpSparse* on the L40S. Compared with 3090 Ti, HSMU-SpGEMM enables even greater speedup than *OpSparse* on L40S.

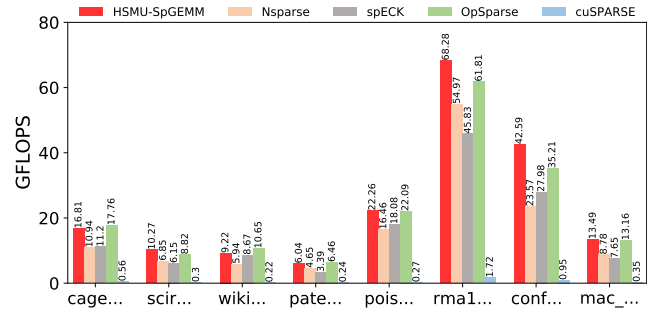


Fig. 16: The performance result for AA^T operation on all eight non-symmetric matrices of the 18 matrices on 3090 Ti.

Fig. 16 illustrates the performance of the AA^T operation on all eight non-symmetric matrices of the 18 representative matrices, where HSMU-SpGEMM still achieves good performance in this scenario.

E. Additional Evaluations

This section is dedicated to evaluating the time breakdown across various stages in HSMU-SpGEMM, as well as the time and space costs associated with the auxiliary mask structure in HSMU-SpGEMM on all 338 matrices.

Fig. 17 plots the time distribution of each stage in HSMU-SpGEMM. HSMU-SpGEMM is roughly divided into four

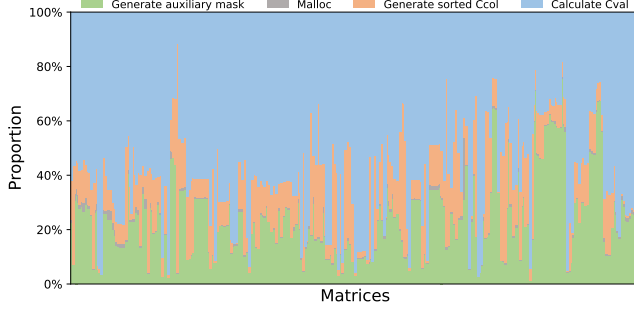


Fig. 17: Execution time breakdown of each stage in HSMU-SpGEMM.

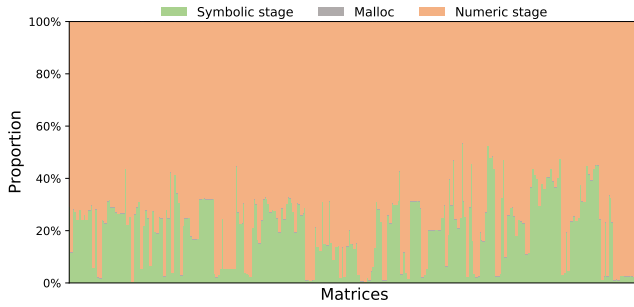


Fig. 18: Execution time breakdown of *OpSparse*.

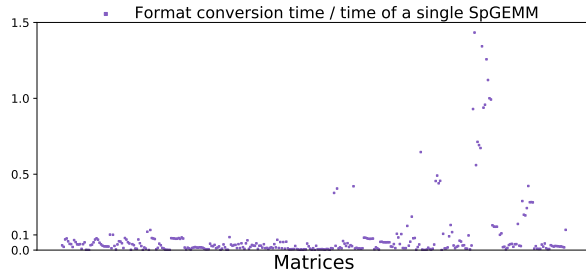


Fig. 19: The ratio of the time overhead of a format conversion to the time overhead of a single SpGEMM.

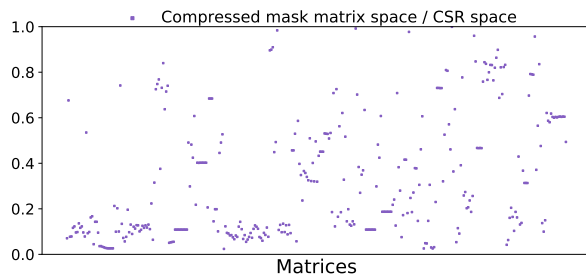


Fig. 20: The ratio of space overhead between the compressed mask matrix and the traditional *CSR* format.

stages: generating the auxiliary mask structure for the matrix C (*Generate auxiliary mask*), allocating memory for the final matrix C (*Malloc*), extracting and sorting the column indices of NZ elements from the auxiliary mask structure (*Generate sorted Ccol*), and computing the NZ values of the final matrix C (*Calculate Cval*). Note that the time overhead associated with computing the NNZ of matrix C and the time overhead of

binning are both included in the *Generate auxiliary mask* part. On most matrices, *Calculate Cval* is the most time-consuming part, accounting for 56.92% of the overall average time. For a small number of matrices, *Generate auxiliary mask* part accounts for a large proportion, and the overall average time is 26.40%. As for *Malloc* and *Generate sorted Ccol* part, their average time proportions are 0.86% and 15.83% respectively. As a comparison, the time breakdown of *OpSparse* is shown in Fig. 18. *Numeric stage* also takes up the majority of the time, averaging 77.4%, while *symbolic stage* accounts for 22.6%.

Fig. 19 illustrates the comparison between the time required for format conversion and the time for a single SpGEMM. The format conversion overhead (from *CSR* to our auxiliary mask structure) is mostly less than one-tenth of the cost of performing a single SpGEMM operation. In many scenarios, such as in Algebraic Multigrid (AMG), SpGEMM is performed iteratively in multiple rounds, where the output of the previous round serves as the input for the next round. Consequently, the overhead incurred during format conversion can be amortized across each SpGEMM. Fig. 20 shows the comparison of space overhead between the compressed mask matrix format and the traditional *CSR* format. The new format utilizes compressed bit masks to record the positions of NZ elements in the matrix, resulting in lower space overhead compared to *CSR* (ratio < 1).

VI. CONCLUSION AND FUTURE WORK

In this paper, we propose a new SpGEMM library with high shared memory utilization, named HSMU-SpGEMM. Unlike traditional hash-based accumulators for SpGEMM, HSMU-SpGEMM avoids the trade-off between GPU shared memory utilization and hash collisions by employing a binary search algorithm-based accumulator. Different from the traditional hash-based accumulators that compute values and column indices of NZ elements simultaneously, HSMU-SpGEMM separates column index calculations from the numeric stage. The column index calculations are done through two versions of generating the sorted column arrays for small-scale matrices and large-scale matrices, respectively. As a result, HSMU-SpGEMM outperforms four existing hash-based libraries (*Nsparse*, *spECK*, *OpSparse*, and *cuSPARSE*), and achieves both high utilization of shared memory and performance improvement. In our future research, we plan to investigate the potential of applying HSMU-SpGEMM in real-world scenarios such as accelerating deep neural networks execution, given its high utilization of shared memory and remarkable performance.

ACKNOWLEDGMENTS

The authors would like to thank all anonymous reviewers for their insightful comments and suggestions, and all reviewers for evaluating and providing constructive feedbacks on HSMU-SpGEMM's artifacts.

REFERENCES

- [1] John R Gilbert, Steve Reinhardt, and Viral B Shah. High-performance graph algorithms from parallel sparse matrices. In *International Workshop on Applied Parallel Computing*, pages 260–269. Springer, 2006.
- [2] Ariful Azad, Georgios A Pavlopoulos, Christos A Ouzounis, Nikos C Kyrpides, and Aydin Buluç. Hipmcl: a high-performance parallel implementation of the markov clustering algorithm for large-scale networks. *Nucleic acids research*, 46(6):e33–e33, 2018.
- [3] Oguz Selvitopi, Md Taufique Hussain, Ariful Azad, and Aydin Buluç. Optimizing high performance markov clustering for pre-exascale architectures. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 116–126. IEEE, 2020.
- [4] Allison H Baker, Todd Gamblin, Martin Schulz, and Ulrike Meier Yang. Challenges of scaling algebraic multigrid across modern multicore architectures. In *2011 IEEE International Parallel & Distributed Processing Symposium (IPDPS)*, pages 275–286. IEEE, 2011.
- [5] Grey Ballard, Christopher Siefert, and Jonathan Hu. Reducing communication costs for sparse matrix multiplication within algebraic multigrid. *SIAM Journal on Scientific Computing*, 38(3):C203–C231, 2016.
- [6] Michael M Wolf, Mehmet Deveci, Jonathan W Berry, Simon D Hammond, and Sivasankaran Rajamanickam. Fast linear algebra-based triangle counting with kokkoskernels. In *2017 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–7. IEEE, 2017.
- [7] Abdurrahman Yaşar, Sivasankaran Rajamanickam, Jonathan Berry, Michael Wolf, Jeffrey S Young, and Ümit V Çatalyürek. Linear algebra-based triangle counting via fine-grained tasking on heterogeneous environments: (update on static graph challenge). In *2019 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–4. IEEE, 2019.
- [8] Fred G Gustavson. Two fast algorithms for sparse matrices: Multiplication and permuted transposition. *ACM Transactions on Mathematical Software (TOMS)*, 4(3):250–269, 1978.
- [9] Julien Demouth. Sparse matrix-matrix multiplication on the gpu. In *Proceedings of the GPU technology conference (GTC)*, volume 3, 2012.
- [10] Yusuke Nagasaka, Akira Nukada, and Satoshi Matsuoka. High-performance and memory-saving sparse general matrix-matrix multiplication for nvidia pascal gpu. In *2017 46th International Conference on Parallel Processing (ICPP)*, pages 101–110. IEEE, 2017.
- [11] Mehmet Deveci, Christian Trott, and Sivasankaran Rajamanickam. Multithreaded sparse matrix-matrix multiplication for many-core and gpu architectures. *Parallel Computing*, 78:33–46, 2018.
- [12] Mathias Parger, Martin Winter, Daniel Mlakar, and Markus Steinberger. speck: accelerating gpu sparse matrix-matrix multiplication through lightweight analysis. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOPP)*, pages 362–375, 2020.
- [13] Zhaoyang Du, Yijin Guan, Tianchan Guan, Dimin Niu, Linyong Huang, Hongzhong Zheng, and Yuan Xie. Opsparse: A highly optimized framework for sparse general matrix multiplication on gpus. *IEEE Access*, 10:85960–85974, 2022.
- [14] Rakshith Kunchum, Ankur Chaudhry, Aravind Sukumaran-Rajam, Qingpeng Niu, Israt Nisa, and P Sadayappan. On improving performance of sparse matrix-matrix multiplication on gpus. In *Proceedings of the International Conference on Supercomputing (ICS)*, pages 1–11, 2017.
- [15] Felix Gremse, Andreas Hofer, Lars Ole Schwen, Fabian Kiessling, and Uwe Naumann. Gpu-accelerated sparse matrix-matrix multiplication by iterative row merging. *SIAM Journal on Scientific Computing*, 37(1):C54–C71, 2015.
- [16] Weifeng Liu and Brian Vinter. An efficient gpu general sparse matrix-matrix multiplication for irregular data. In *2014 IEEE 28th International Parallel and Distributed Processing Symposium (IPDPS)*, pages 370–381. IEEE, 2014.
- [17] Nathan Bell and Michael Garland. Cusp: Generic parallel algorithms for sparse matrix and graph computations. *Version 0.3. 0*, 35, 2012.
- [18] Martin Winter, Daniel Mlakar, Rhaleb Zayer, Hans-Peter Seidel, and Markus Steinberger. Adaptive sparse matrix-matrix multiplication on the gpu. In *Proceedings of the 24th symposium on principles and practice of parallel programming (PPOPP)*, pages 68–81, 2019.
- [19] Julien Demouth. Sparse matrix-matrix multiplication on the gpu. Technical report, NVIDIA, 1993.
- [20] NVIDIA. NVIDIA cuSPARSE library. <https://developer.nvidia.com/cuSPARSE>, 2023.
- [21] Zhuo Tian, Shuai Yang, and Changyou Zhang. Accelerating sparse general matrix-matrix multiplication for nvidia volta gpu and hygon dcu. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, pages 329–330, 2023.
- [22] Timothy A Davis and Yifan Hu. The university of florida sparse matrix collection. *ACM Transactions on Mathematical Software (TOMS)*, 38(1):1–25, 2011.
- [23] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. A systematic survey of general sparse matrix-matrix multiplication. *ACM Computing Surveys*, 55(12):1–36, 2023.
- [24] Pham Nguyen Quang Anh, Rui Fan, and Yonggang Wen. Balanced hashing and efficient gpu sparse general matrix-matrix multiplication. In *Proceedings of the 2016 International Conference on Supercomputing*, pages 1–12, 2016.
- [25] André R Brodtkorb, Trond R Hagen, Christian Schulz, and Geir Hasle. Gpu computing in discrete optimization. part i: Introduction to the gpu. *EURO journal on transportation and logistics*, 2(1-2):129–157, 2013.
- [26] Putt Sakdhnagool, Amit Sabne, and Rudolf Eigenmann. Regdem: Increasing gpu performance via shared memory register spilling. *arXiv preprint arXiv:1907.02894*, 2019.
- [27] Maurice Herlihy, Nir Shavit, and Moran Tzafrir. Hopscotch hashing. In *Distributed Computing: 22nd International Symposium, DISC 2008, Arcachon, France, September 22-24, 2008. Proceedings 22*, pages 350–364. Springer, 2008.
- [28] Duane Merrill and Michael Garland. Merge-based sparse matrix-vector multiplication (spmv) using the csr storage format. *Acm Sigplan Notices*, 51(8):1–2, 2016.
- [29] NVIDIA. cusparsespGEMM_compute. <https://docs.nvidia.com/cuda/cusparses/index.html>, 2024.
- [30] Zhiyao Li, Jiaxiang Li, Taijie Chen, Dimin Niu, Hongzhong Zheng, Yuan Xie, and Mingyu Gao. Spada: accelerating sparse matrix multiplication with adaptive dataflow. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, pages 747–761, 2023.
- [31] Jinkwon Kim, Myeongjae Jang, Haejin Nam, and Soontae Kim. Harp: Hardware-based pseudo-tiling for sparse matrix multiplication accelerator. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1148–1162, 2023.
- [32] Yang Wang, Chen Zhang, Zhiqiang Xie, Cong Guo, Yunxin Liu, and Jingwen Leng. Dual-side sparse tensor core. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 1083–1095. IEEE, 2021.
- [33] Zhekai Zhang, Hanrui Wang, Song Han, and William J Dally. Sparch: Efficient architecture for sparse matrix multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 261–274. IEEE, 2020.
- [34] Yuyao Niu, Zhengyang Lu, Haonan Ji, Shuhui Song, Zhou Jin, and Weifeng Liu. Tilespgemm: A tiled algorithm for parallel sparse general matrix-matrix multiplication on gpus. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOPP)*, pages 90–106, 2022.
- [35] Guowei Zhang, Nithya Attaluri, Joel S Emer, and Daniel Sanchez. Gamma: Leveraging gustavson’s algorithm to accelerate sparse matrix multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 687–701, 2021.

A. Artifact Appendix

A.1 Abstract

This section summarizes the artifact evaluation for this work. First, we provide the check-list for this artifact. Next, we describe the directory structure for the code. Finally, the installation, experiment workflow, and evaluation illustrate how to use the artifact to reproduce results and extend the implementation.

A.2 Artifact check-list (meta-information)

- **Program:** Cuda code.
- **Compilation:** NVCC
- **Data set:** SuiteSparse Matrix Collection (<https://sparse.tamu.edu/>).
- **Run-time environment:** Require isolated server as experiments sensitive to resource contention, adding performance variability to tail-latency.
- **Hardware:** Server class Intel CPU and NVIDIA GPU
- **Execution:** Sole user.
- **Output:** Experiments produce console and text files. Expected output either in paper or code repository.
- **How much disk space required (approximately)?:** 100 GB
- **How much time is needed to prepare workflow (approximately)?:** 1 hour
- **How much time is needed to complete experiments (approximately)?:** 1 hour
- **Publicly available?:** Yes

A.3 Description

A.3.1 How to access

Code for this paper can be accessed by cloning the public GitHub repository: <https://github.com/wuminqaq/HSMU-SpGEMM>

A.3.2 Hardware dependencies

To reproduce results found in this paper, we suggest using commensurate hardware. This includes an 12th Gen Intel(R) Core(TM) i9-12900K CPU and an NVIDIA GeForce RTX 3090 Ti. In the absence of this hardware, we suggest using a similar setup, such as a server with a many-core CPU and an NVIDIA GPU with a compute capability of 8.6.

A.3.3 Software dependencies

The artifact has been tested on Ubuntu 22.04.4, GCC 9.5.0, CUDA 11.4; For script execution, the following python libraries are required: Pandas, Matplotlib, Numpy.

A.3.4 Data sets

HSMU-SpGEMM currently supports input matrix files with the *.mtx* extension. All of the matrices we tested can be downloaded from the SuiteSparse Matrix Collection (<https://sparse.tamu.edu/>).

A.4 Installation

Download the source code locally from <https://github.com/wuminqaq/HSMU-SpGEMM>. Download the required software, or use Docker (download the corresponding version of cuda and gcc images from <https://hub.docker.com/r/nvidia/cuda/tags?page=3>)

A.5 Experiment workflow

- **Compile:** Compile with the commands `$cd evaluation/script` followed by `$make` to generate an executable file named *test*. Run HSMU-SpGEMM with the following command:
`$ ./test <path/to/matrix.set>`

- **Download matrix sets:** The 18 representative matrix sets tested in the paper can be downloaded:
From `<HSMU_dir>/evaluation/18representMatrixSet`, run the command:
`$python3 18matrix_download.py`.
The 338 matrices tested in the paper can be downloaded:
From `<HSMU_dir>/evaluation/338MatrixSet`, run the command:
`$python3 338matrix_download.py` (requires 9 hours or more).
- **Simple run example:** From the `<HSMU_dir>/evaluation/script/`, run the command:
`$/simple_run_test.sh`
This script uses the matrix "cant" as a test case to prove that the artifact is available and correct.
- From the `<HSMU_dir>/evaluation/script/`, run Figure 6 experiment (180 minutes or more):
`$/test_threshold_matrix.sh`
- From the `<HSMU_dir>/evaluation/script/`, run Figure 7 experiment:
`$/test_four_extremely_large_matrices.sh`
- From the `<HSMU_dir>/evaluation/script/`, run Figure 9 and Tables 3, 4 experiments (20 minutes or more):
`$/test338matrices.sh`
- From the `<HSMU_dir>/evaluation/script/`, run Figure 10 experiment (2 minutes):
`$/test18matrices.sh`
- From the `<HSMU_dir>/evaluation/script/`, run Figure 11 experiment :
`$/test_peak_memory.sh`
- From the `<HSMU_dir>/evaluation/script/`, run Figure 16 experiment:
`$/testAATmatrices.sh`
- From the `<HSMU_dir>/evaluation/script/`, run Figures 17, 19, and 20 experiments:
`$/additional_evaluation.sh`

A.6 Evaluation and expected results

Each of the artifact evaluation scripts describe in the "Experiment Workflow" produce results for a specific figure in this paper. The bash scripts include Python scripts to format the experiment outputs for specific bars and lines in each Figure. For each matrix tested, the output information is as follows:

lines 1-3 provide basic information about the tested matrix, including its name, number of rows, number of columns, and the number of non-zero elements.

line 4 outputs the loading time of the matrix file (in seconds).

line 5 outputs the format conversion time (in milliseconds) (corresponds to Figure 19).

line 6 outputs the space overhead of the new format (corresponds to Figure 20).

lines 7-11 present the time taken by each stage of HSMU-SpGEMM (in milliseconds) (corresponds to Figure 17).

lines 12-14 provide information about matrix C, including the number of intermediate products, the number of non-zero elements in

C, and the compression ratio.

lines 15-16 output the running time, the overall performance in GFLOPS, and the GFLOPS of the numeric stage.

A.7 Experiment customization

The codebase is organized to facilitate customization and extension. The configuration files in `inc/` allow for easily specifying experiment configurations including whether to count step time, whether to measure the time and space overhead of format conversion, the number of runs, the threshold of using dense accumulators, etc.

A.8 Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>